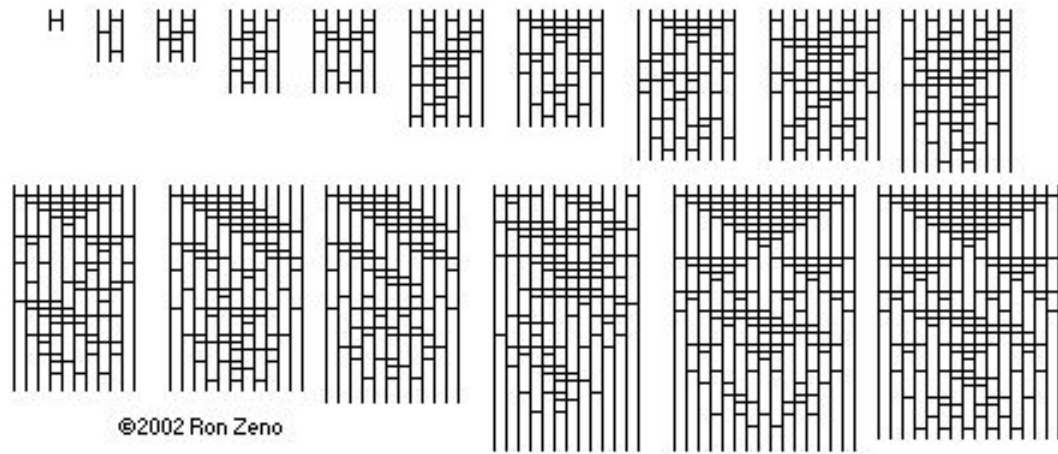




## Lecture 35 (Sorting 4)

# Theoretical Bounds on Sorting

CS61B, Fall 2025 @ UC Berkeley

Slides credit: Josh Hug

Sorting is a foundational problem.

- Obviously useful for putting things in order.
- But can also be used to solve other tasks, sometimes in non-trivial ways.
  - Sorting improves duplicate finding from a naive  $N^2$  to  $N \log N$ .
  - Sorting improves 3SUM from a naive  $N^3$  to  $N^2$ .
- There are many ways to sort an array, each with its own interesting tradeoffs and algorithmic features.

Today we'll discuss the fundamental nature of the sorting problem itself: How hard is it to sort?

## Sorts Summary

|                | Memory           | # Compares                  | Notes                               | Stable? |
|----------------|------------------|-----------------------------|-------------------------------------|---------|
| Heapsort       | $\Theta(1)$      | $\Theta(N \log N)$          | Bad caching (61C)                   | No      |
| Insertion      | $\Theta(1)$      | $\Theta(N^2)$               | Best for almost sorted and $N < 15$ | Yes     |
| Mergesort      | $\Theta(N)$      | $\Theta(N \log N)$          | Fastest stable sort                 | Yes     |
| Quicksort LTHS | $\Theta(\log N)$ | $\Theta(N \log N)$ expected | Fastest sort                        | No      |

This is due to the cost of tracking recursive calls by the computer, and is also an “expected” amount. The difference between  $\log N$  and constant memory is trivial.

You can create a stable Quicksort. However, using unstable partitioning schemes (like Hoare partitioning) and using randomness to avoid bad pivots tend to yield better runtimes.

# Math Problem Warmup

---

Lecture 35, CS61B, Fall 2025

## Math Problem Warmup

Theoretical Bounds on Sorting

- Simple Bounds for TUCS (the ultimate comparison sort)
- Puppy Cat Dog ( $N = 3$ ,  $N = 4$ )
- Puppy Cat Dog for Any  $N$
- The Sorting Lower Bound

Consider the functions  $N!$  and  $(N/2)^{N/2}$

Is  $N! \in \Omega((N/2)^{N/2})$ ? Prove your answer.

- Recall that  $\in \Omega$  can be informally be interpreted to mean  $\geq$
- In other words, does factorial grow at least as quickly as  $(N/2)^{N/2}$ ?

Consider the functions  $N!$  and  $(N/2)^{N/2}$

Is  $N! \in \Omega((N/2)^{N/2})$ ? Prove your answer.

$10!$

- $10 * 9 * 8 * 7 * 6 * \dots * 1$

$5^5$

- $5 * 5 * 5 * 5 * 5$

$N! > (N/2)^{N/2}$ , for large  $N$ , therefore  $N! \in \Omega((N/2)^{N/2})$

Given:  $N! > (N/2)^{N/2}$ , which we used to prove our answer to the previous problem.

Show that  $\log(N!) \in \Omega(N \log N)$ .

- Recall:  $\log$  means an unspecified base.

Given that  $N! > (N/2)^{N/2}$

Show that  $\log(N!) \in \Omega(N \log N)$ .

We have that  $N! > (N/2)^{N/2}$

- Taking the log of both sides, we have that  $\log(N!) > \log((N/2)^{N/2})$ .
- Bringing down the exponent we have that  $\log(N!) > N/2 \log(N/2)$ .
- Discarding the unnecessary constant, we have  $\log(N!) \in \Omega(N \log (N/2))$ .
- From there, we have that  $\log(N!) \in \Omega(N \log N)$ .

Since  $\log(N/2)$  is the same thing asymptotically as  $\log(N)$ .

In other words,  $\log(N!)$  grows at least as quickly as  $N \log N$ .

In the previous problem, we showed that  $\log(N!) \in \Omega(N \log N)$ .

Now show that  $N \log N \in \Omega(\log(N!))$ .

Show that  $N \log N \in \Omega(\log(N!))$

Proof:

- $\log(N!) = \log(N) + \log(N-1) + \log(N-2) + \dots + \log(1)$
- $N \log N = \log(N) + \log(N) + \log(N) + \dots \log(N)$
- Therefore  $N \log N \in \Omega(\log(N!))$

Given:

- $N \log N \in \Omega(\log(N!))$
- $\log(N!) \in \Omega(N \log N)$

Which of the following can we say?

- A.  $N \log N \in \Theta(\log N!)$
- B.  $\log N! \in \Theta(N \log N)$
- C. Both A and B
- D. Neither

Given:

- $N \log N \in \Omega(\log(N!))$
- $\log(N!) \in \Omega(N \log N)$

Informally:  $N \log N \geq \log(N!)$



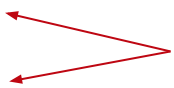
Informally:  $\log(N!) \geq N \log N$



Which of the following can we say?

- A.  $N \log N \in \Theta(\log N!)$
- B.  $\log N! \in \Theta(N \log N)$
- C. Both A and B**
- D. Neither

Informally:  $N \log N = \log(N!)$



## Summary

---

We've shown that  $\log(N!) \in \Theta(N \log N)$ .

- In other words, these two functions grow at the same rate asymptotically.

As for why we did this, we will see in a little while...

# Simple Bounds for TUCS (the ultimate comparison sort)

---

Lecture 35, CS61B, Fall 2025

Math Problem Warmup

## Theoretical Bounds on Sorting

- **Simple Bounds for TUCS (the ultimate comparison sort)**
- Puppy Cat Dog ( $N = 3$ ,  $N = 4$ )
- Puppy Cat Dog for Any  $N$
- The Sorting Lower Bound

We have shown several sorts to require  $\Theta(N \log N)$  worst case time.

- Can we build a better sorting algorithm?

By comparison sort, I mean that it uses e.g. the `compareTo` method in Java to make decisions.

Let the ultimate comparison sort (TUCS) be the asymptotically fastest possible comparison sorting algorithm, possibly yet to be discovered, and let  $R(N)$  be its worst case runtime.

Give the best  $\Omega$  and  $O$  bounds you can for  $R(N)$ .

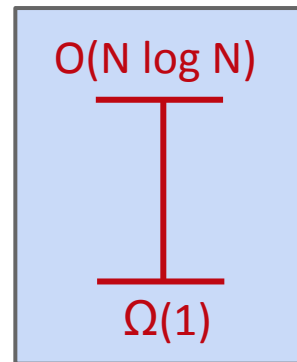
It might seem strange to give  $\Omega$  and  $O$  bounds for an algorithm whose details are completely unknown, but you can, I promise!

We have shown several sorts to require  $\Theta(N \log N)$  worst case time.

- Can we build a better sorting algorithm? By comparison sort, I mean that it uses e.g. the `compareTo` method in Java to make decisions.

Let the ultimate comparison sort (TUCS) be the asymptotically fastest possible comparison sorting algorithm, possibly yet to be discovered, and let  $R(N)$  be its worst case runtime.

- Worst case run-time of TUCS,  $R(N)$  is  $O(N \log N)$ .
  - Obvious: Mergesort is  $\Theta(N \log N)$  so  $R(N)$  can't be worse!
- Worst case run-time of TUCS,  $R(N)$  is  $\Omega(1)$ .
  - Obvious: Problem doesn't get easier with  $N$ .
  - Can we make a stronger statement than  $\Omega(1)$ ?

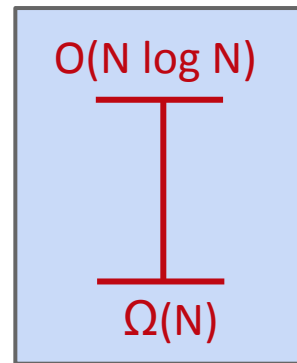


TUCS Worst  
Case  $\Theta$  Runtime

## Sorting

Let TUCS be the asymptotically fastest possible comparison sorting algorithm, possibly yet to be discovered.

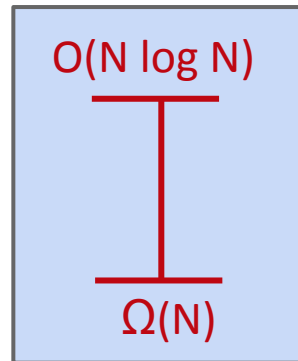
- Worst case run-time of TUCS,  $R(N)$  is  $O(N \log N)$ . Why?
- Worst case run-time of TUCS,  $R(N)$  is also  $\Omega(N)$ .
  - Have to at least look at every item.



TUCS Worst  
Case  $\Theta$  Runtime

We know that TUCS “lives” between  $N$  and  $N \log N$ .

- Worst case asymptotic runtime of TUCS is between  $\Theta(N)$  and  $\Theta(N \log N)$ .
- Can we make an even stronger statement on the lower bound?
  - With a clever argument, yes (as we’ll see soon see).
    - Spoiler alert: It will turn out to be  $\Omega(N \log N)$
  - This lower bound means that across the infinite space of all possible ideas that any human might ever have for sorting using sequential comparisons, NONE has a worst case runtime that is better than  $\Theta(N \log N)$ .



TUCS Worst  
Case  $\Theta$  Runtime

# Puppy Cat Dog ( $N = 3$ , $N = 4$ )

---

Lecture 35, CS61B, Fall 2025

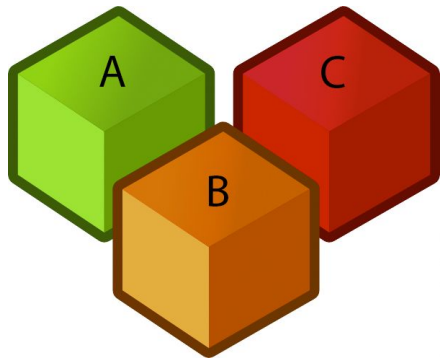
Math Problem Warmup

## Theoretical Bounds on Sorting

- Simple Bounds for TUCS (the ultimate comparison sort)
- **Puppy Cat Dog ( $N = 3$ ,  $N = 4$ )**
- Puppy Cat Dog for Any  $N$
- The Sorting Lower Bound

# The Game of Puppy, Cat, Dog

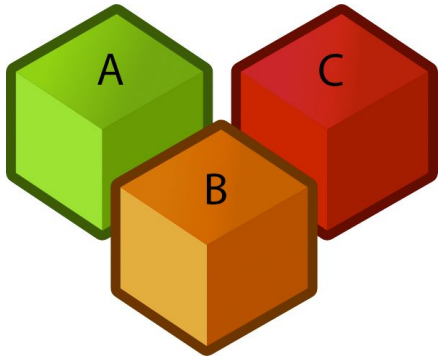
Suppose we have a puppy, a cat, and a dog, each in an opaque soundproof box labeled A, B, and C. We want to figure out which is which using a scale.



# The Game of Puppy, Cat, Dog

Suppose we have a puppy, a cat, and a dog, each in an opaque soundproof box labeled A, B, and C. We want to figure out which is which using a scale.

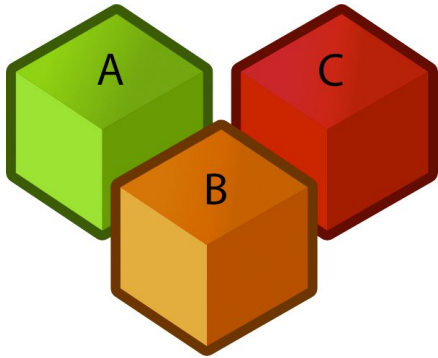
|         |         |  |                 |
|---------|---------|--|-----------------|
| $a < b$ | $b < c$ |  | Which is which? |
| Yes     | Yes     |  |                 |
|         |         |  |                 |
|         |         |  |                 |



# The Game of Puppy, Cat, Dog

Suppose we have a puppy, a cat, and a dog, each in an opaque soundproof box labeled A, B, and C. We want to figure out which is which using a scale.

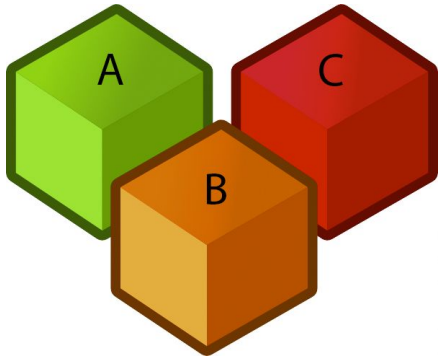
|         |         |  |                                              |
|---------|---------|--|----------------------------------------------|
| $a < b$ | $b < c$ |  | Which is which?                              |
| Yes     | Yes     |  | a: puppy, b: cat, c: dog (sorted order: abc) |
| No      | No      |  |                                              |
|         |         |  |                                              |



# The Game of Puppy, Cat, Dog

Suppose we have a puppy, a cat, and a dog, each in an opaque soundproof box labeled A, B, and C. We want to figure out which is which using a scale.

| $a < b$ | $b < c$ |  | Which is which?                              |
|---------|---------|--|----------------------------------------------|
| Yes     | Yes     |  | a: puppy, b: cat, c: dog (sorted order: abc) |
| No      | No      |  | c: puppy, b: cat, a: dog (sorted order: cba) |
|         |         |  |                                              |



Suppose we have a puppy, a cat, and a dog, each in an opaque soundproof box labeled A, B, and C. We want to figure out which is which using a scale.

| a < b | b < c |  | Which is which?                              |
|-------|-------|--|----------------------------------------------|
| Yes   | Yes   |  | a: puppy, b: cat, c: dog (sorted order: abc) |
| No    | No    |  | c: puppy, b: cat, a: dog (sorted order: cba) |
| Yes   | No    |  |                                              |

Which is which?

- 1. a: puppy, b: cat, c: dog (sorted order: abc)
- 2. a: puppy, c: cat, b: dog (sorted order: acb)
- 3. c: puppy, a: cat, b: dog (sorted order: cab)
- 4. c: puppy, b: cat, a: dog (sorted order: cba)

# The Game of Puppy, Cat, Dog

Suppose we have a puppy, a cat, and a dog, each in an opaque soundproof box labeled A, B, and C. We want to figure out which is which using a scale.

|       |       |  |                                              |
|-------|-------|--|----------------------------------------------|
| a < b | b < c |  | Which is which?                              |
| Yes   | Yes   |  | a: puppy, b: cat, c: dog (sorted order: abc) |
| No    | No    |  | c: puppy, b: cat, a: dog (sorted order: cba) |
| Yes   | No    |  |                                              |

Which is which? How do we resolve the ambiguity?

- 1. a: puppy, b: cat, c: dog (sorted order: abc)
- 2. **a: puppy, c: cat, b: dog (sorted order: acb)**
- 3. **c: puppy, a: cat, b: dog (sorted order: cab)**
- 4. c: puppy, b: cat, a: dog (sorted order: cba)

a?      c?      b  
c?      a?      b

## The Game of Puppy, Cat, Dog

Suppose we have a puppy, a cat, and a dog, each in an opaque soundproof box labeled A, B, and C. We want to figure out which is which using a scale.

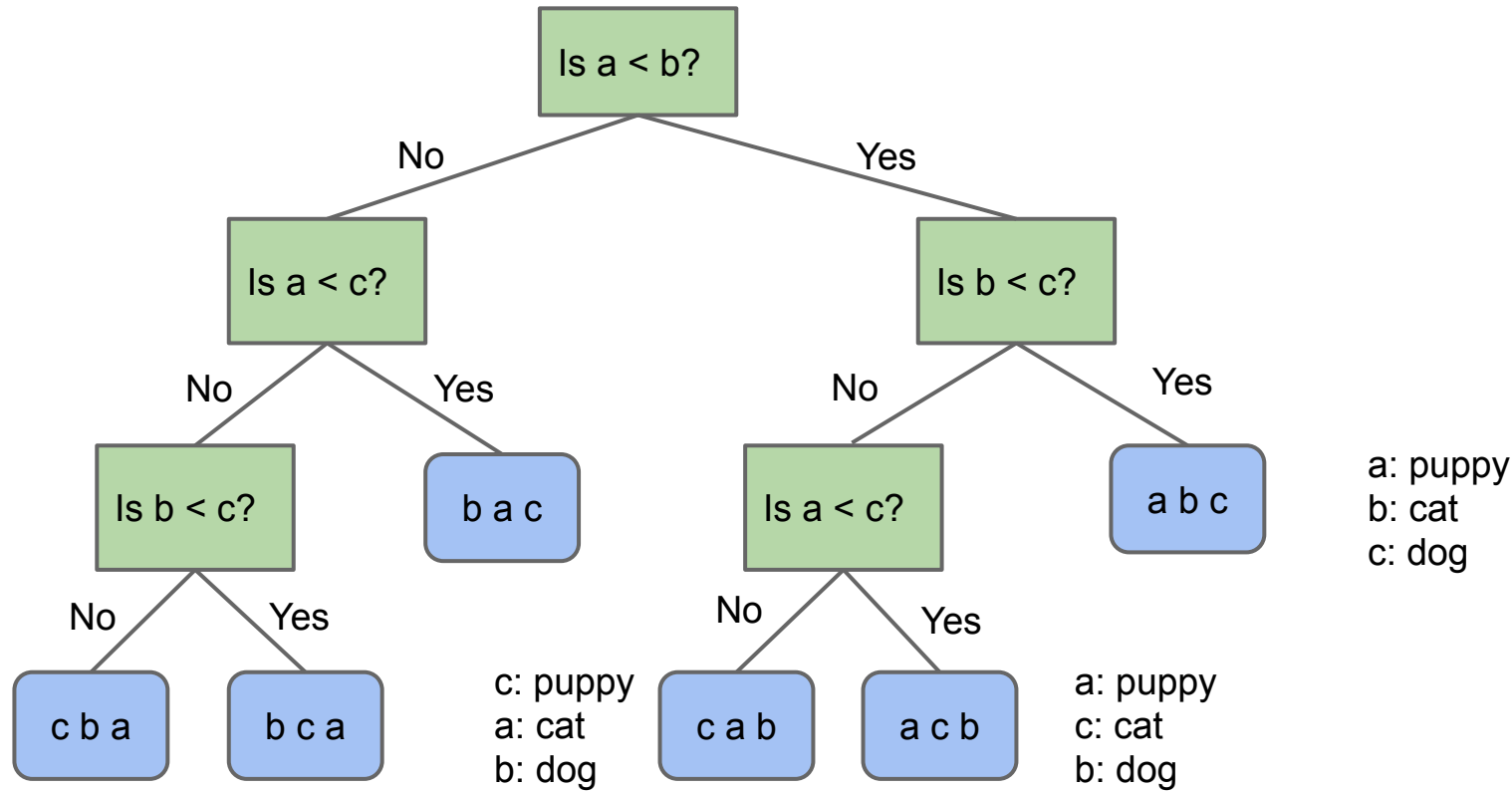
| $a < b$ | $b < c$ | $a < c?$ | Which is which?                              |
|---------|---------|----------|----------------------------------------------|
| Yes     | Yes     | N/A      | a: puppy, b: cat, c: dog (sorted order: abc) |
| No      | No      | N/A      | c: puppy, b: cat, a: dog (sorted order: cba) |
| Yes     | No      | Yes      | a: puppy, c: cat, b: dog (sorted order: acb) |

Which is which? How do we resolve the ambiguity? Ask if  $a < c$ .

1. a: puppy, b: cat, c: dog (sorted order: abc)
2. **a: puppy, c: cat, b: dog (sorted order: acb)**
3. **c: puppy, a: cat, b: dog (sorted order: cab)**
4. c: puppy, b: cat, a: dog (sorted order: cba)

# Puppy, Cat, Dog - A Graphical Picture for N = 3

The full decision tree for puppy, cat, dog:



How many questions would you need to ask to definitely solve the “puppy, cat, dog, walrus” problem?

- A. 3
- B. 4
- C. 5
- D. 6

How many questions would you need to ask to definitely solve the “puppy, cat, dog, walrus” problem?

- A. 3
- B. 4
- C. 5**
- D. 6

Proof:

- If  $N=4$ , how many permutations?  $4! = 24$ 
  - For  $N=3$ :  $3!=6$
- So we need a binary tree with 24 leaves.
  - How many levels minimum?  $\lg(24) = 4.58$ , so 5 is the minimum.
  - $\lg$  just means  $\log_2$  (log base 2)

# Puppy Cat Dog for Any N

---

Lecture 35, CS61B, Fall 2025

Arrays.sort in Java

Math Problem Warmup

## Theoretical Bounds on Sorting

- Simple Bounds for TUCS (the ultimate comparison sort)
- Puppy Cat Dog ( $N = 3$ ,  $N = 4$ )
- **Puppy Cat Dog for Any N**
- The Sorting Lower Bound

Sound of Sorting

How many questions would you need to ask to definitely solve the generalized “puppy, cat, dog” problem for  $N$  items?

- Give your answer in big Omega notation.

Hint: For  $N=4$ , we said the answer was 5 based on the following argument:

- Decision tree needs  $4! = 24$  leaves.
- So we need  $\lg(24)$  rounded up levels or 5.

How many questions would you need to ask to definitely solve the generalized “puppy, cat, dog” problem for  $N$  items?

Answer:  $\Omega(\log(N!))$

Hint: For  $N$ , we have the following argument:

- Decision tree needs  $N!$  leaves.
- So we need  $\lg(N!)$  rounded up levels, which is  $\Omega(\log(N!))$

Finding an optimal decision tree for the generalized version of puppy, cat, dog (e.g.  $N=6$ : puppy, cat, dog, monkey, walrus, elephant) is an open problem in mathematics.

- (To my knowledge) Best known trees known for  $N=1$  through 15 and  $N=22$ :
  - For more, see: <http://oeis.org/A036604>

Deriving a sequence of yes/no questions to identify puppy, cat, dog is hard. An alternate approach to solving the puppy, cat, dog problem:

- Sort the boxes using any generic sorting algorithm.
  - Leftmost box is puppy.
  - Middle box is cat.
  - Right box is dog.

Why do we care about these (no doubt adorable) critters?

A solution to the sorting problem also provides a solution to puppy, cat, dog.

- In other words, puppy, cat, dog **reduces** to sorting.
- Thus, any lower bound on difficulty of puppy, cat, dog must ALSO apply to sorting.

Physics analogy: Climbing a hill with your legs (CAHWYL) is one way to solve the problem of getting up a hill (GUAH).

- Any lower bound on energy to GUAH must also apply to CAHWYL.
- Example bound: Takes  $m \cdot g \cdot h$  energy to climb hill, so using legs to climb the hill takes at least  $m \cdot g \cdot h$  energy.

# The Sorting Lower Bound

---

Lecture 35, CS61B, Fall 2025

Math Problem Warmup

## Theoretical Bounds on Sorting

- Simple Bounds for TUCS (the ultimate comparison sort)
- Puppy Cat Dog ( $N = 3$ ,  $N = 4$ )
- Puppy Cat Dog for Any  $N$
- **The Sorting Lower Bound**

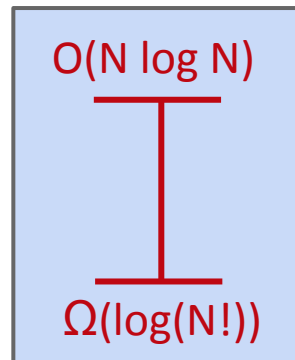
## Sorting Lower Bound

We have a lower bound on puppy, cat, dog, namely that it takes  $\Omega(\log(N!))$  comparisons to solve such a puzzle in the worst case.

Since sorting with comparisons can be used to solve puppy, cat, dog, then sorting also takes  $\Omega(\log(N!))$  comparisons in the worst case.

Or in other words:

- Any sorting algorithm using comparisons, no matter how clever, must use at least  $k = \lg(N!)$  compares to find the correct permutation. So even TUCS takes at least  $\lg(N!)$  comparisons.
- $\lg(N!)$  is trivially  $\Omega(\log(N!))$ , so TUCS must take  $\Omega(\log(N!))$  time.
- So, how does  $\log(N!)$  compare to  $N \log N$ ?



TUCS Worst  
Case  $\Theta$  Runtime

Earlier, we showed that  $\log(N!) \in \Omega(N \log N)$  using the proof below.

- In other words,  $\log(N!)$  grows at least as quickly as  $N \log N$ .

Proof from earlier that  $\log(N!) \in \Omega(N \log N)$ :

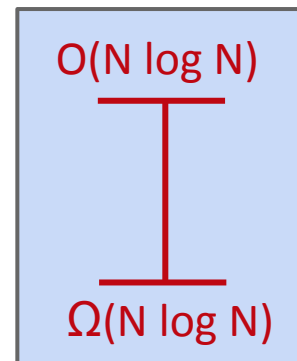
- We know that  $N! \geq (N/2)^{N/2}$ .
- Taking the log of both sides, we have that  $\log(N!) \geq \log((N/2)^{N/2})$ .
- Bringing down the exponent we have that  $\log(N!) \geq N/2 \log(N/2)$ .
- Discarding unnecessary constants, we have  $\log(N!) \in \Omega(N \log N)$

Recall that changing base is just multiplying by a constant.

## The Sorting Lower Bound (Finally)

Since TUCS is  $\Omega(\lg N!)$  and  $\lg N!$  is  $\Omega(N \log N)$ , we have that **TUCS is  $\Omega(N \log N)$** .

**Any comparison based sort requires at least order  $N \log N$  comparisons in its worst case.**



TUCS Worst  
Case  $\Theta$  Runtime

## The Sorting Lower Bound (Finally)

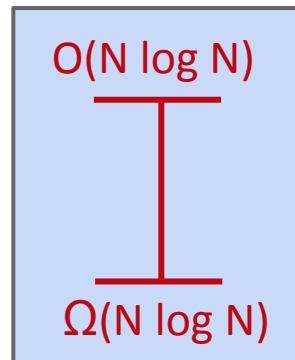
Since TUCS is  $\Omega(\lg N!)$  and  $\lg N!$  is  $\Omega(N \log N)$ , we have that **TUCS is  $\Omega(N \log N)$** .

**Any comparison based sort requires at least order  $N \log N$  comparisons in its worst case.**

Proof summary:

- Puppy, cat, dog is  $\Omega(\lg N!)$ , i.e. requires  $\lg N!$  comparisons.
- TUCS can solve puppy, cat, dog, and thus takes  $\Omega(\lg N!)$  compares.
- $\lg(N!)$  is  $\Omega(N \log N)$ 
  - This was because  $N!$  is  $\Omega(N/2)^{N/2}$

Informally:  $\text{TUCS} \geq \text{puppy, cat, dog} \geq \log N! \geq N \log N$



TUCS Worst  
Case  $\Theta$  Runtime

|                | Memory           | # Compares                  | Notes                               | Stable? |
|----------------|------------------|-----------------------------|-------------------------------------|---------|
| Heapsort       | $\Theta(1)$      | $\Theta(N \log N)$          | Bad caching (61C)                   | No      |
| Insertion      | $\Theta(1)$      | $\Theta(N^2)$               | Best for almost sorted and $N < 15$ | Yes     |
| Mergesort      | $\Theta(N)$      | $\Theta(N \log N)$          | Fastest stable sort                 | Yes     |
| Quicksort LTHS | $\Theta(\log N)$ | $\Theta(N \log N)$ expected | Fastest sort                        | No      |

The punchline:

- Our best sorts have achieved absolute asymptotic optimality.
  - Mathematically impossible to sort using fewer comparisons.
  - Note: Randomized quicksort is only probabilistically optimal, but the probability is extremely high for even modest  $N$ . Are you worried about quantum teleportation? Then don't worry about Quicksort.

## Next Time...

---

Today we proved that any sort that uses comparisons has runtime  $\Omega(N \log N)$ .

Next time we'll discuss how we can sort in  $\Theta(N)$  time.

- Not impossible, just can't compare anything while we sort!

# Citations

---

Title image: <http://www.angelfire.com/blog/ronz/Articles/999SortingNetworksReferen.html>

Cubes: [http://www.clker.com/cliparts/6/9/3/2/1197122947130754155jean\\_victor\\_balin\\_Cubes.svg.hi.png](http://www.clker.com/cliparts/6/9/3/2/1197122947130754155jean_victor_balin_Cubes.svg.hi.png)

Scale: <http://www.clipartbest.com/cliparts/94T/bAe/94TbAejig.png>

Puppy: [http://assets.nydailynews.com/polopoly\\_fs/1.1245686!/img/httpImage/image.jpg\\_gen/derivatives/article\\_970/afp-cute-puppy.jpg](http://assets.nydailynews.com/polopoly_fs/1.1245686!/img/httpImage/image.jpg_gen/derivatives/article_970/afp-cute-puppy.jpg)

Cat: <http://animalia-life.com/cat.html>

Dog: <http://animalia-life.com/dogs.html>

